



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Names:	Molar, Jabez C. Oabel, Edmarc Justin C.	Date:	August 30, 2026
Program:	BS CPE		

- A. Learning Outcomes:**
- At the end of this laboratory, students should be able to:**
1. Differentiate Strings, Lists, Tuples, and Dictionaries.
 2. Apply appropriate Python data structures in software design.
 3. Develop a simple console-based application using collections.
 4. Demonstrate teamwork through collaborative software development.
 5. Analyze the advantages and limitations of different data structures.

B. Introduction:

Python provides several built-in data structures used to organize and manage data efficiently.

1. Strings

Strings are used to store text data such as:

```
name = "Juan Dela Cruz"
course = "BS Computer Engineering"
```

- Use Strings For:
- Student Name
 - Course
 - Labels and Output Messages

2. Lists

Lists are used to store multiple subjects.

```
subjects = [
"Programming",
"Calculus",
"Physics"
]
```

- Use Lists For:
- Subject Management
 - Displaying Subject Information

3. Tuples

Tuples store fixed information that should not be changed.

```
student_id = (2026, 1001)
```

- Use Tuples For:
- Student ID
 - Year and Student Number

4. Dictionary

Store complete record:

```
student = {
"Name": name,
"Course": course,
"Student ID": student_id,
"Subjects": subjects
}
```

Used as the primary data container.

Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

C. Laboratory Scenario:

A university needs a simple **Student Information Management System**. 2 Members

The system should:

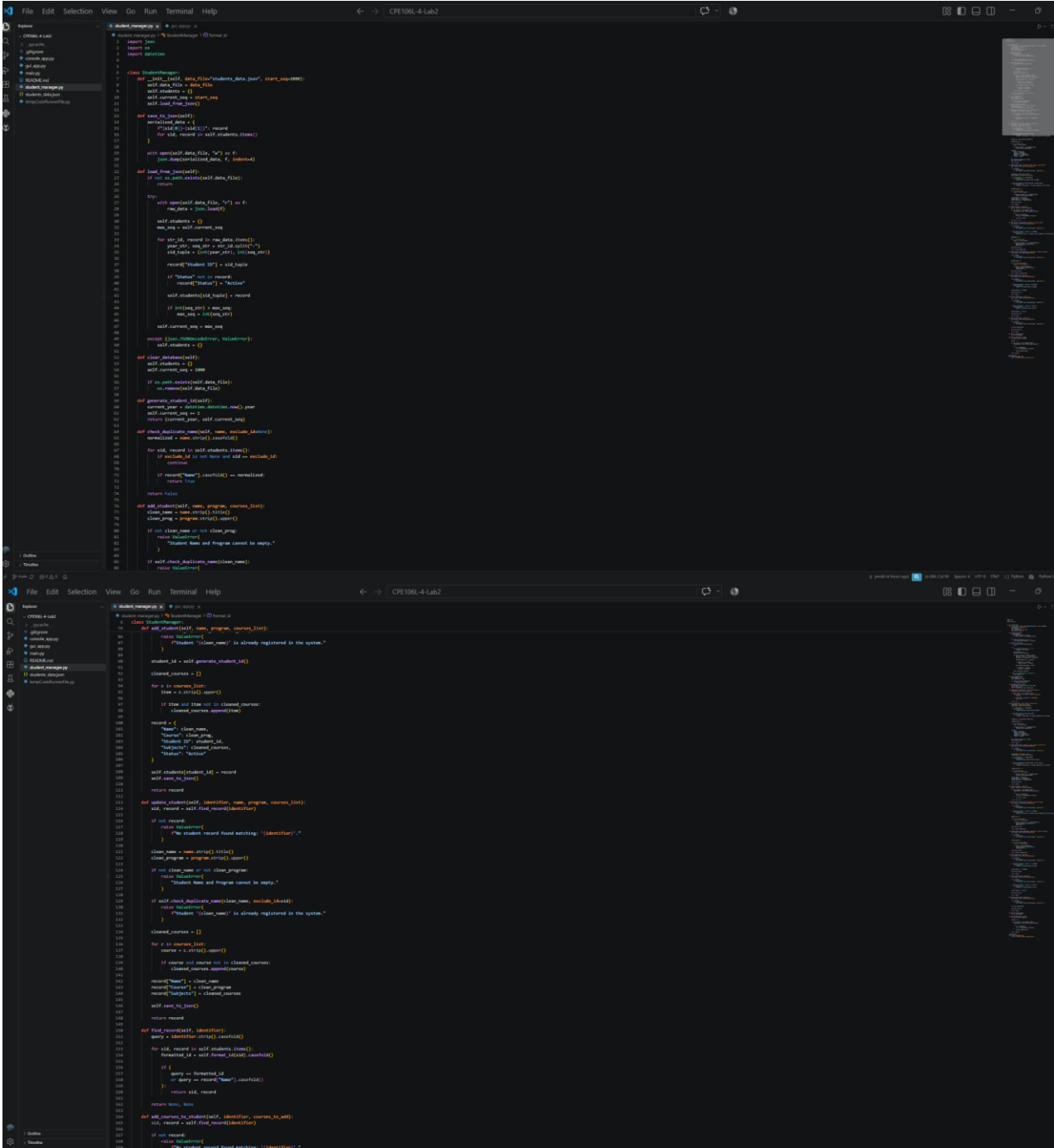
- 1. Store student names.
- 2. Manage enrolled subjects.
- 3. Generate a unique student identifier.
- 4. Save student details using dictionaries.

GITHUB REPOSITORY LINK: <https://github.com/jxnobl/CPE106L-4-Lab2.git>

(Please include the following: 1. Screenshots of the code 2. Screenshots of the output)

Screenshots of Code (Name of Members) – MOLAR, OABEL

student_manager.py



```
1 # student_manager.py
2
3 # Importing modules
4 import os
5 import sys
6 import random
7
8 # Class StudentManager
9 class StudentManager:
10     def __init__(self, data_file="students_data.json", start_year=1990):
11         self.data_file = data_file
12         self.start_year = start_year
13         self.students = {}
14         self.current_id = 0
15         self.current_year = start_year
16         self.load_data()
17
18     def load_data(self):
19         """Load student data from a JSON file"""
20         if os.path.exists(self.data_file):
21             with open(self.data_file, "r") as f:
22                 data = json.load(f)
23                 for student in data:
24                     self.add_student(student["name"], student["program"], student["courses"])
25         else:
26             # Create a new file if it doesn't exist
27             with open(self.data_file, "w") as f:
28                 json.dump([], f)
29
30     def add_student(self, name, program, courses):
31         """Add a new student to the system"""
32         if name in self.students:
33             raise ValueError(f"Student {name} is already registered in the system.")
34
35         # Generate a unique student ID
36         student_id = self.generate_student_id()
37
38         # Create a record for the student
39         record = {
40             "name": name,
41             "program": program,
42             "courses": courses,
43             "id": student_id,
44             "year": self.current_year
45         }
46
47         self.students[student_id] = record
48         self.current_year += 1
49         self.save_data()
50
51     def generate_student_id(self):
52         """Generate a unique student ID"""
53         self.current_id += 1
54         return self.current_id
55
56     def save_data(self):
57         """Save student data to a JSON file"""
58         with open(self.data_file, "w") as f:
59             json.dump(list(self.students.values()), f)
60
61     def find_record(self, identifier):
62         """Find a student record by ID or name"""
63         if identifier.isdigit():
64             # Search by ID
65             student_id = int(identifier)
66             if student_id in self.students:
67                 return self.students[student_id]
68             else:
69                 raise ValueError(f"Student with ID {identifier} not found.")
70         else:
71             # Search by name
72             for student in self.students.values():
73                 if student["name"] == identifier:
74                     return student
75             raise ValueError(f"Student with name {identifier} not found.")
76
77     def add_courses_to_student(self, student_id, courses):
78         """Add courses to a student's record"""
79         student = self.find_record(student_id)
80         if student:
81             student["courses"] = courses
82             self.save_data()
83         else:
84             raise ValueError(f"Student with ID {student_id} not found.")
85
86     def remove_student(self, identifier):
87         """Remove a student from the system"""
88         student = self.find_record(identifier)
89         if student:
90             del self.students[student["id"]]
91             self.save_data()
92         else:
93             raise ValueError(f"Student with ID {identifier} not found.")
94
95     def list_students(self):
96         """List all students in the system"""
97         return list(self.students.values())
98
99     def __str__(self):
100         return f"Student Manager with {len(self.students)} students."
101
102 # Main function
103 def main():
104     manager = StudentManager()
105
106     # Add a new student
107     name = input("Enter student name: ")
108     program = input("Enter student program: ")
109     courses = input("Enter student courses (comma-separated): ")
110     courses = courses.split(",")
111
112     manager.add_student(name, program, courses)
113
114     # Find a student record
115     identifier = input("Enter student ID or name: ")
116     record = manager.find_record(identifier)
117
118     if record:
119         print(f"Student found: {record}")
120     else:
121         print("Student not found.")
122
123     # Add courses to a student
124     student_id = input("Enter student ID: ")
125     new_courses = input("Enter new courses (comma-separated): ")
126     new_courses = new_courses.split(",")
127
128     manager.add_courses_to_student(student_id, new_courses)
129
130     # Remove a student
131     identifier = input("Enter student ID or name: ")
132     manager.remove_student(identifier)
133
134     # List all students
135     students = manager.list_students()
136     for student in students:
137         print(student)
138
139 if __name__ == "__main__":
140     main()
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```
File Edit Selection View Go Run Terminal Help
CPE106L-4 Lab2

# StudentManager.py
# @author: [Your Name]
# @date: 2024-01-15
# @description: A class to manage student records.

class StudentManager:
    def __init__(self):
        self.students = {}
        self.records = {}

    def add_student(self, student_id, name, email):
        """Add a new student to the system.

        Args:
            student_id (int): The unique identifier for the student.
            name (str): The student's name.
            email (str): The student's email address.

        Returns:
            bool: True if the student was added successfully, False otherwise.
        """
        if student_id in self.students:
            return False
        self.students[student_id] = name
        self.records[student_id] = {}
        return True

    def add_course(self, student_id, course_id, grade):
        """Add a course record for a student.

        Args:
            student_id (int): The student's ID.
            course_id (int): The course ID.
            grade (float): The grade earned in the course.

        Returns:
            bool: True if the record was added successfully, False otherwise.
        """
        if student_id not in self.records:
            return False
        if course_id in self.records[student_id]:
            return False
        self.records[student_id][course_id] = grade
        return True

    def remove_student(self, student_id):
        """Remove a student from the system.

        Args:
            student_id (int): The student's ID.

        Returns:
            bool: True if the student was removed successfully, False otherwise.
        """
        if student_id not in self.students:
            return False
        del self.students[student_id]
        del self.records[student_id]
        return True

    def get_all_students(self):
        """Return a list of all students in the system.

        Returns:
            list: A list of tuples containing student ID and name.
        """
        return [(id, name) for id, name in self.students.items()]

    def get_all_records(self):
        """Return a list of all records in the system.

        Returns:
            list: A list of tuples containing student ID, course ID, and grade.
        """
        results = []
        for student_id, records in self.records.items():
            for course_id, grade in records.items():
                results.append((student_id, course_id, grade))
        return results

    def __str__(self):
        """Return a string representation of the StudentManager object.

        Returns:
            str: A string containing the number of students and records.
        """
        return f"StudentManager: {len(self.students)} students, {len(self.records)} records"

# Example usage
if __name__ == "__main__":
    manager = StudentManager()
    manager.add_student(1, "John Doe", "john.doe@example.com")
    manager.add_course(1, 101, 85)
    manager.add_course(1, 102, 78)
    manager.add_student(2, "Jane Smith", "jane.smith@example.com")
    manager.add_course(2, 101, 92)
    manager.add_course(2, 103, 88)
    manager.remove_student(1)
    print(manager.get_all_students())
    print(manager.get_all_records())
    print(manager)
```

Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

console_app.py

```
File Edit Selection View Go Run Terminal Help
C:\Users\user> python console_app.py
1 from student_manager import StudentManager
2
3 def main_console_app():
4     manager = StudentManager()
5     print("\n===== WELCOME =====")
6     print("\n===== REGISTER =====")
7     print("\n===== ADD =====")
8     print("\n===== REMOVE =====")
9     print("\n===== SEARCH =====")
10    print("\n===== CLEAR =====")
11    print("\n===== RESET =====")
12    print("\n===== MENU =====")
13
14    while True:
15        choice = input("Enter choice (1-4): ")
16        if choice == "1":
17            register_console_app()
18        elif choice == "2":
19            add_console_app()
20        elif choice == "3":
21            remove_console_app()
22        elif choice == "4":
23            search_console_app()
24        elif choice == "5":
25            clear_console_app()
26        elif choice == "6":
27            reset_console_app()
28        elif choice == "7":
29            menu_console_app()
30        else:
31            print("Invalid choice. Please select a valid option (1-7).")
32
33    print("\n===== END =====")
34
35 if __name__ == "__main__":
36     main_console_app()
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Screenshots of Output

```
=====
CPE106L - APPLICATION ENTRY LAUNCHER
=====
1. Launch Console-Based System
2. Launch Graphical User Interface (GUI)
3. Reset All JSON Data
4. Exit Application

Enter choice (1-4): 1

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 3

=====
ACTIVE STUDENT DIRECTORY
=====

ID Code   : 2026-1001
Name      : Edmarc Justin C. Oabel
Program   : BS COMPUTER ENGINEERING
Courses   : 7 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1
-----

ID Code   : 2026-1002
Name      : Jabez C. Molar
Program   : BS COMPUTER ENGINEERING
Courses   : 7 Enrolled -> CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-4, CPE117-4
-----

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 1
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```
[CANCELLED] Database reset aborted.

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 2

=====
MANAGE ENROLLED COURSES
=====
Enter Student ID or Name: Justin De Leon

Student Found   : Justin De Leon (2026-1003)
Current Courses : CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1

Choose Action:
1. Add Course(s)
2. Drop / Delete Course(s)
Select (1-2): 2
Enter Courses to Drop (comma-separated): IE103-1

[SUCCESS] Dropped 1 course(s).
Updated Enrolled: CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 3

=====
ACTIVE STUDENT DIRECTORY
=====

ID Code   : 2026-1001
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```

ID Code : 2026-1001
Name : Edmarc Justin C. Oabel
Program : BS COMPUTER ENGINEERING
Courses : 7 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1
-----

ID Code : 2026-1002
Name : Jabez C. Molar
Program : BS COMPUTER ENGINEERING
Courses : 7 Enrolled -> CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-4, CPE117-4
-----

ID Code : 2026-1003
Name : Justin De Leon
Program : BS COMPUTER ENGINEERING
Courses : 6 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L
-----

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 6
Exiting console interface...

=====
CPE106L - APPLICATION ENTRY LAUNCHER
=====
1. Launch Console-Based System
2. Launch Graphical User Interface (GUI)
3. Reset All JSON Data
4. Exit Application

Enter choice (1-4): █

```

Laboratory Report 2
Strings, Lists, Tuples, and Dictionaries

- 5. Basic GUI using Tkinter. No need to be creative yet, just create a functional GUI and embed the system.
2 Members

(Please include the following: 1. Screenshots of the code 2. Screenshots of the output)

Screenshots of Code (Name of Members) - MOLAR, OABEL

gui_app.py

```
1 def new_gui_manager():
2     import tkinter as tk
3     from tkinter import messagebox, ttk
4     from student_manager import StudentManager
5
6     manager = StudentManager()
7
8     # Colors
9     BG_COLOR = "#f0f0f0"
10    CANV_COLOR = "#ffffff"
11    PRIMARY_COLOR = "#4682b4"
12    SECONDARY_COLOR = "#808080"
13    ALERT_COLOR = "#ff4500"
14    TEXT_COLOR = "#000000"
15    MENU_TEXT = "#808080"
16    SUCCESS_COLOR = "#4682b4"
17    SUCCESS_COLOR = "#4682b4"
18    DANGER_COLOR = "#ff4500"
19    WARNING_COLOR = "#ff4500"
20
21
22
23
24 def refresh_table():
25     for item in tree.get_children():
26         tree.delete(item)
27
28     for id, record in manager.get_all_students().items():
29         formatted_id = manager.format_id(id)
30         courses = record.get("subjects", [])
31
32         tree.insert(
33             "",
34             tk.END,
35             values=[
36                 formatted_id,
37                 record["name"],
38                 record["course"],
39                 len(courses),
40                 " ".join(courses) if courses else "None",
41                 record.get("status", "Active")
42             ]
43         )
44
45 def clear_fields():
46     entry_name.delete(0, tk.END)
47     entry_program.delete(0, tk.END)
48     entry_courses.delete(0, tk.END)
49
50     entry_name.focus()
51
52     status_label.config(
53         text="Ready for student registration."
54     )
55
56
57 def register_student_gui():
58     name = entry_name.get().strip()
59     program = entry_program.get().strip()
60     raw_courses = entry_courses.get().strip()
61
62     courses = []
63     if raw_courses:
64         for c in raw_courses.split(","):
65             if c.strip():
66                 courses.append(c.strip())
67
68     if not name or not program:
69         messagebox.showwarning(
70             "Incomplete Information",
71             "Please enter both the student's Name and Program."
72         )
73         return
74
75     try:
76         record = manager.add_student(
77             name,
78             program,
79             courses
80         )
81         formatted_id = manager.format_id(
82             record["id"]
83         )
84
85         refresh_table()
86         clear_fields()
87
88         status_label.config(
89             text=f"Student registered successfully - ID: {formatted_id}"
90         )
91
92         messagebox.showinfo(
93             "Registration Successful",
94             f"Record initialized successfully.\nID: {formatted_id}\nName: {record['name']}\nProgram: {record['course']}"
95         )
96     except ValueError as e:
97         messagebox.showerror(
98             "Registration Error",
99             str(e)
100        )
101
102 def get_selected_student():
103     selected = tree.selection()
104
105     if not selected:
106         messagebox.showwarning(
107             "No Student Selected",
108             "Please select a student from the directory first."
109         )
110         return None
111
112     return tree.item(selected[0], "values")
113
114 def edit_student_gui():
115     value = get_selected_student()
116
117     if not value:
118         return
119
120     student_id = value[0]
```

main.py

Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```
main.py X
main.py > main
1 import sys
2 from student_manager import StudentManager
3 from console_app import run_console_app
4 from gui_app import run_gui_app
5
6 def main():
7     manager = StudentManager()
8
9     while True:
10         print("\n=====")
11         print(" CPE106L - APPLICATION ENTRY LAUNCHER ")
12         print("=====")
13         print("1. Launch Console-Based System")
14         print("2. Launch Graphical User Interface (GUI)")
15         print("3. Reset All JSON Data")
16         print("4. Exit Application")
17
18         choice = input("\nEnter choice (1-4): ").strip()
19
20         if choice == "1":
21             run_console_app(manager)
22         elif choice == "2":
23             run_gui_app(manager)
24         elif choice == "3":
25             confirm = input("Reset stored data file? (y/n): ").strip().lower()
26             if confirm == 'y':
27                 manager.clear_database()
28                 print("JSON storage file reset successfully.")
29         elif choice == "4":
30             print("\nShutting down application. Goodbye!")
31             sys.exit(0)
32         else:
33             print("\nInvalid choice. Please input 1, 2, 3, or 4.")
34
35 if __name__ == "__main__":
36     main()
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Screenshots of Output

University Academic Records System

UNIVERSITY ACADEMIC RECORDS SYSTEM

Student Registration & Academic Directory

Student Registration

Enter the student's academic information below.

Student Name

Justin De Leon

UPDATE STUDENT

Program

BS COMPUTER ENGINEERING

CLEAR FIELDS

Courses

CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L

CANCEL EDIT

Active Student Directory

Registered student records

ID Code	Name	Program	Courses	Enrolled Courses	Status
2026-1001	Edmarc Justin C. Oabel	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active
2026-1002	Jabez C. Molar	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-	Active
2026-1003	Justin De Leon	BS COMPUTER ENGINEERING	6	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active

EDIT STUDENT

DROP COURSE

DROP STUDENT

RESTORE STUDENT

DELETE RECORD

Drop Course

Drop Course(s)

Justin De Leon (2026-1003)

Enter course(s) to drop:

Example: CPE106L-4, MATH165

DROP COURSE

Editing student 2026-1003 — modify the fields and click UPDATE STUDENT.

University Academic Records System

UNIVERSITY ACADEMIC RECORDS SYSTEM

Student Registration & Academic Directory

Student Registration

Enter the student's academic information below.

Student Name

REGISTER STUDENT

Program

CLEAR FIELDS

Courses

Active Student Directory

Registered student records

ID Code	Name	Program	Courses	Enrolled Courses	Status
2026-1001	Edmarc Justin C. Oabel	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active
2026-1002	Jabez C. Molar	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-	Active
2026-1003	Justin De Leon	BS COMPUTER ENGINEERING	6	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Dropped

EDIT STUDENT

DROP COURSE

DROP STUDENT

RESTORE STUDENT

DELETE RECORD

Student 2026-1003 has been marked as DROPPED.



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

D. Findings, Observations, and Comments:

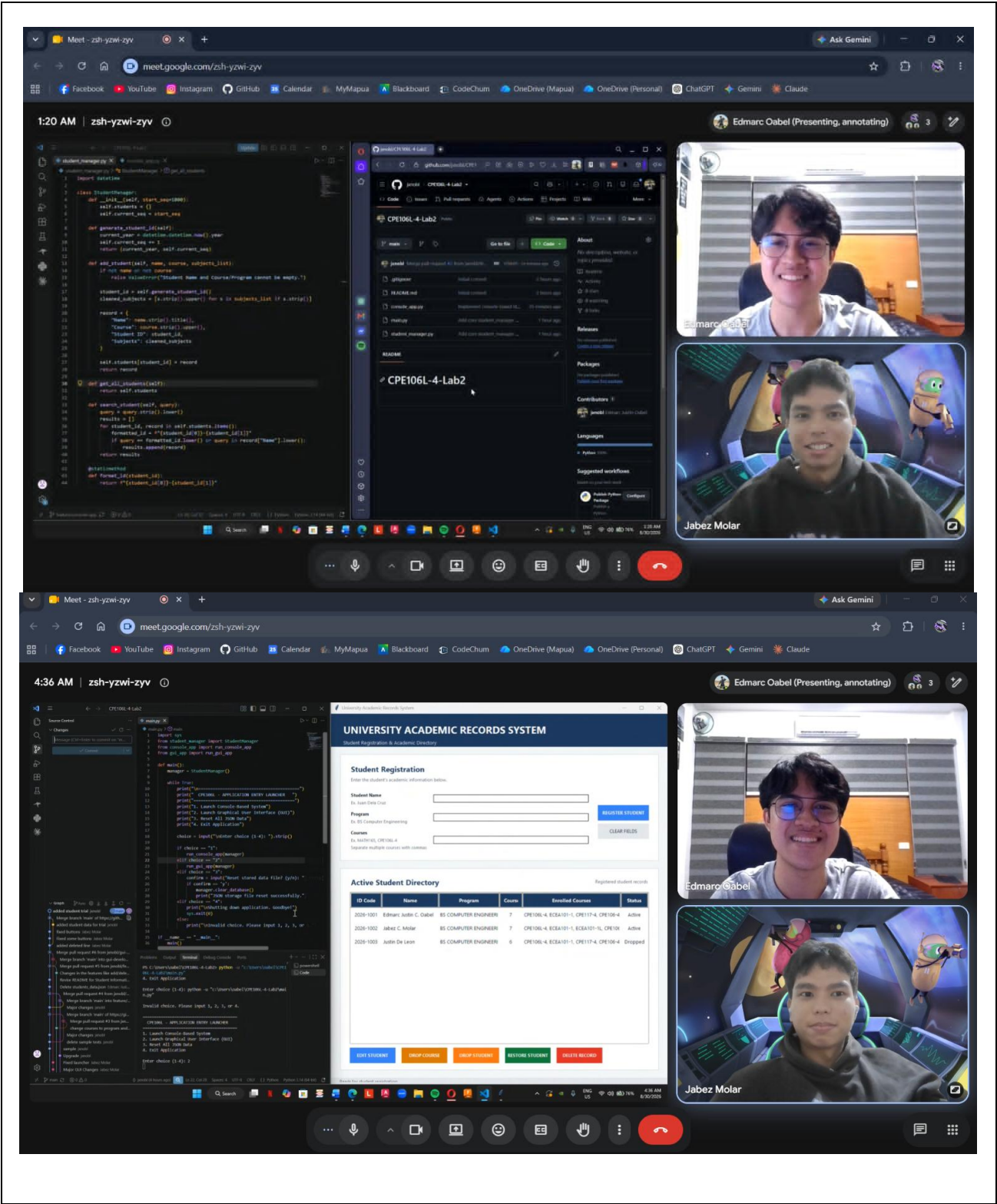
Guide Questions:

- 1. What are your Group’s realizations in this laboratory?
- 2.

Through this activity, our group realized how GitHub and VS Code can be used for collaboration and software development. We learned how to use push, pull, merge, and branches to manage our code and work together without affecting each other’s work. The activity also taught us the purpose of these features and how they can help organize our projects, track changes, and avoid conflicts. We also realized the importance of communication when working with groupmates, especially when deciding who would work on specific parts of the project. Using branches allowed us to work on our assigned tasks separately and combine our work when it was ready.

Laboratory Report 2
Strings, Lists, Tuples, and Dictionaries

PICTURE/PROOF OF DOING THE ACTIVITY



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED

Rating		Signature	
		Date	